

ЛАБОРАТОРНАЯ РАБОТА № 3

«Введение в алгоритмы и структуры данных. Исследование возможностей
коллекций в языке программирования Java»

Студент(ка): Зырянова Татьяна Евгеньевна

Группа: РИМ – 150950

Предмет: Программирование на Java

Репозиторий ДЗ: https://github.com/ZyrT12/java-core_2026-2027

Цель: приобретение навыков работы с рекурсивными методами, знакомство с динамическими структурами данных, приобретение навыков создания и использования простейшей динамической структуры, исследование возможностей коллекций в языке программирования Java, сравнение коллекций по времени выполнения операций, знакомство с реализацией коллекций.

Описание задачи: в данной лабораторной работе необходимо разработать набор программ на языке Java, направленных на изучение рекурсивных алгоритмов, динамических структур данных и стандартных коллекций. В ходе выполнения работы требуется повторить примеры по рекурсии из первого раздела, реализовать самостоятельные задания из раздела 3, создать программы для работы с односвязным списком, выполнить сравнение коллекций ArrayList, LinkedHashSet и LinkedHashMap по варианту 14, ответить на теоретический вопрос и решить две задачи с сайта Timus.

Ход выполнения работы

1. Создать приложение для демонстрации примера 1 из раздела 1: вывести последовательность значений по правилу $x = 2 * x + 1$ при условии $0 \leq x < 20$.

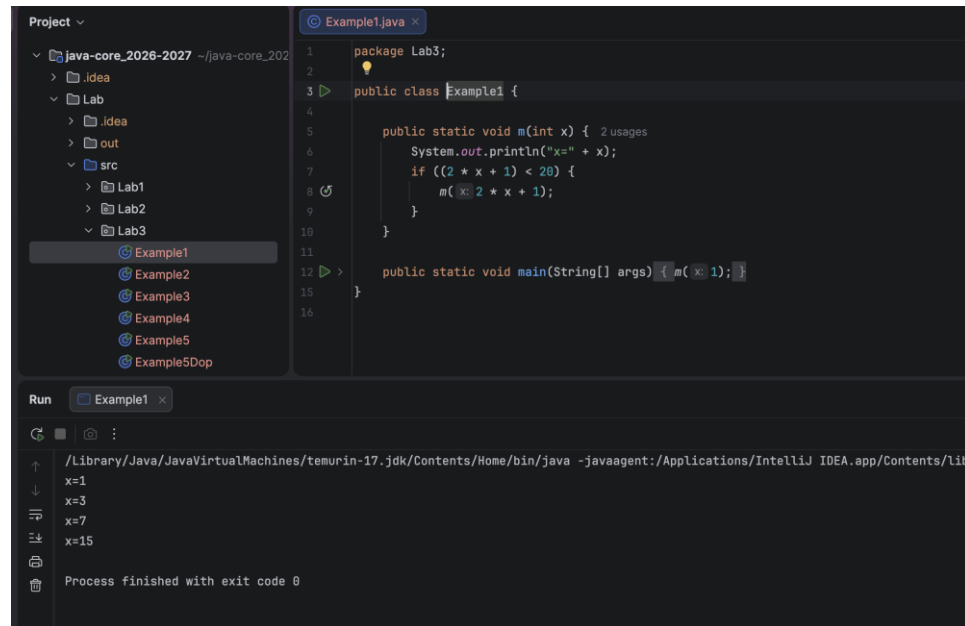


Рисунок 1. Результат работы программы Example1

2. Вывести последовательность, представленную в предыдущем примере, в обратном порядке.

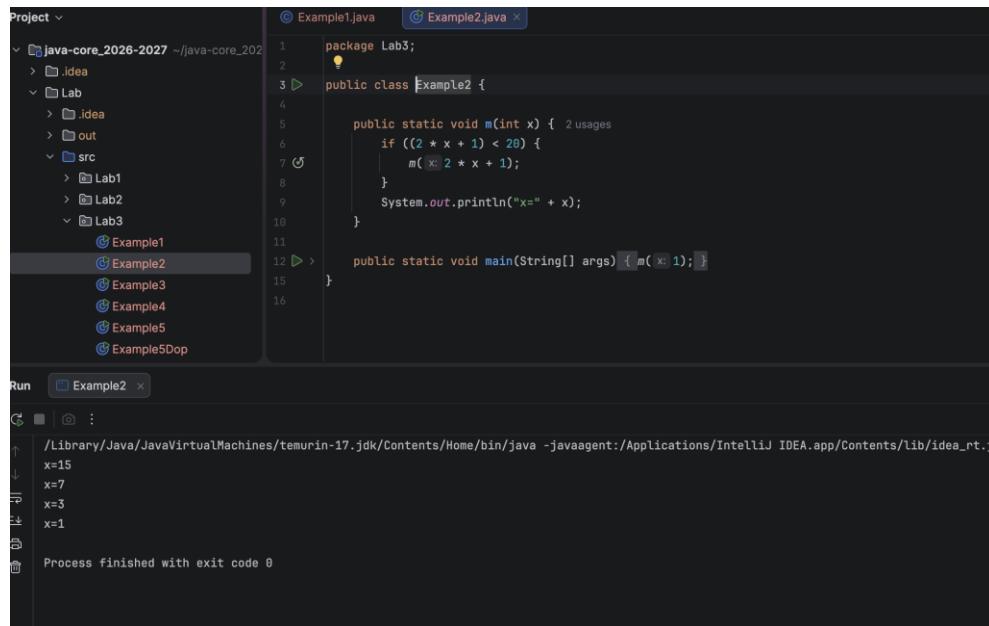
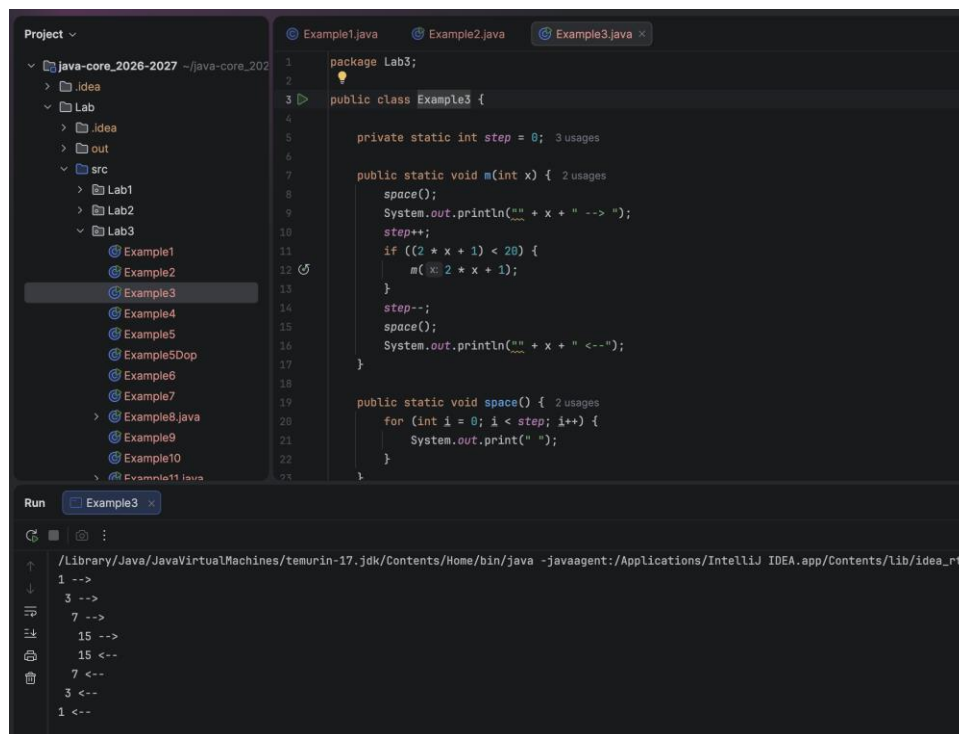


Рисунок 2. Результат работы программы Example2

3. Код вывода параметра перед вхождением в рекурсивный вызов и после него, показан на рисунке 3.



The screenshot shows the IntelliJ IDEA interface. On the left, the Project Explorer displays a directory structure with 'Lab3' containing 'Example3'. The main editor shows the code for 'Example3.java':

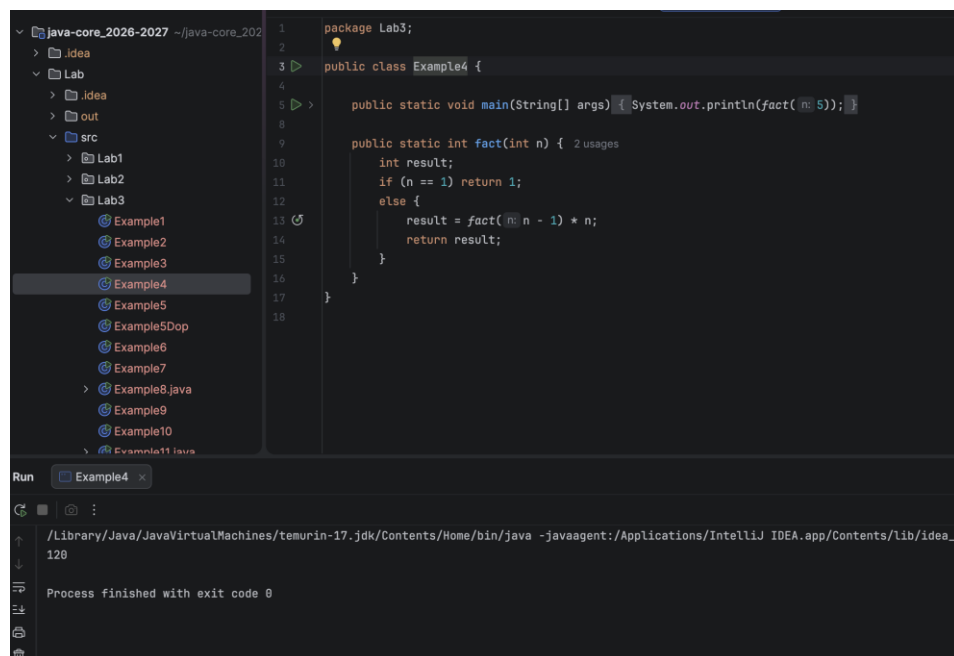
```
1 package Lab3;
2
3 public class Example3 {
4
5     private static int step = 0; // 3 usages
6
7     public static void m(int x) { // 2 usages
8         space();
9         System.out.println("x + " --> ");
10        step++;
11        if ((2 * x + 1) < 20) {
12            m(2 * x + 1);
13        }
14        step--;
15        space();
16        System.out.println("x + " <--");
17    }
18
19    public static void space() { // 2 usages
20        for (int i = 0; i < step; i++) {
21            System.out.print(" ");
22        }
23    }
24 }
```

The Run window at the bottom shows the output of the program:

```
/Library/Java/JavaVirtualMachines/temurin-17.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/Contents/lib/idea_rt...
1 -->
3 -->
7 -->
15 -->
15 <--
7 <--
3 <--
1 <--
```

Рисунок 3. Результат работы программы Example3

4. Вычислить факториал числа n с использованием рекурсии.



The screenshot shows the IntelliJ IDEA interface. On the left, the Project Explorer displays a directory structure with 'Lab3' containing 'Example4'. The main editor shows the code for 'Example4.java':

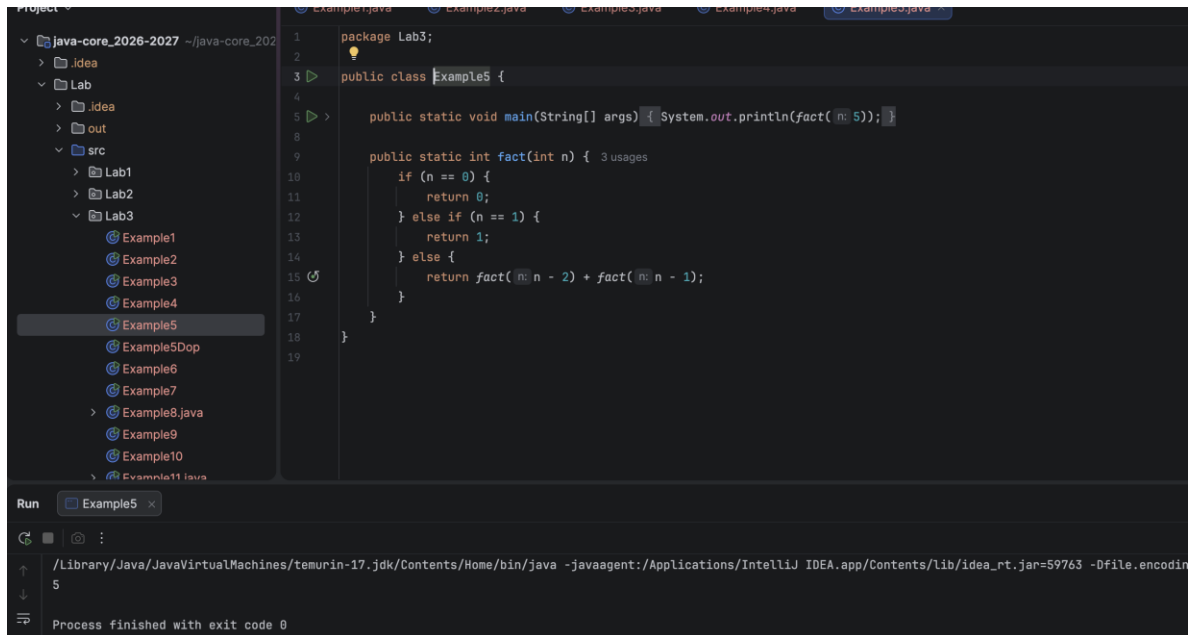
```
1 package Lab3;
2
3 public class Example4 {
4
5     public static void main(String[] args) { System.out.println(fact(5)); }
6
7     public static int fact(int n) { // 2 usages
8         int result;
9         if (n == 1) return 1;
10        else {
11            result = fact(n - 1) * n;
12            return result;
13        }
14    }
15 }
16
17
18 }
```

The Run window at the bottom shows the output of the program:

```
/Library/Java/JavaVirtualMachines/temurin-17.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/Contents/lib/idea_r...
120
Process finished with exit code 0
```

Рисунок 4. Результат работы программы Example4

5. Вывести число Фибоначчи, заданное его номером в последовательности.
Дополнительно вывести последовательность обхода дерева рекурсивных вызовов.



The screenshot shows the IntelliJ IDEA interface. On the left, the Project Explorer displays the file structure of 'java-core_2026-2027', with 'Example5' selected under 'Lab3'. The main editor window shows the code for 'Example5.java':

```
package Lab3;

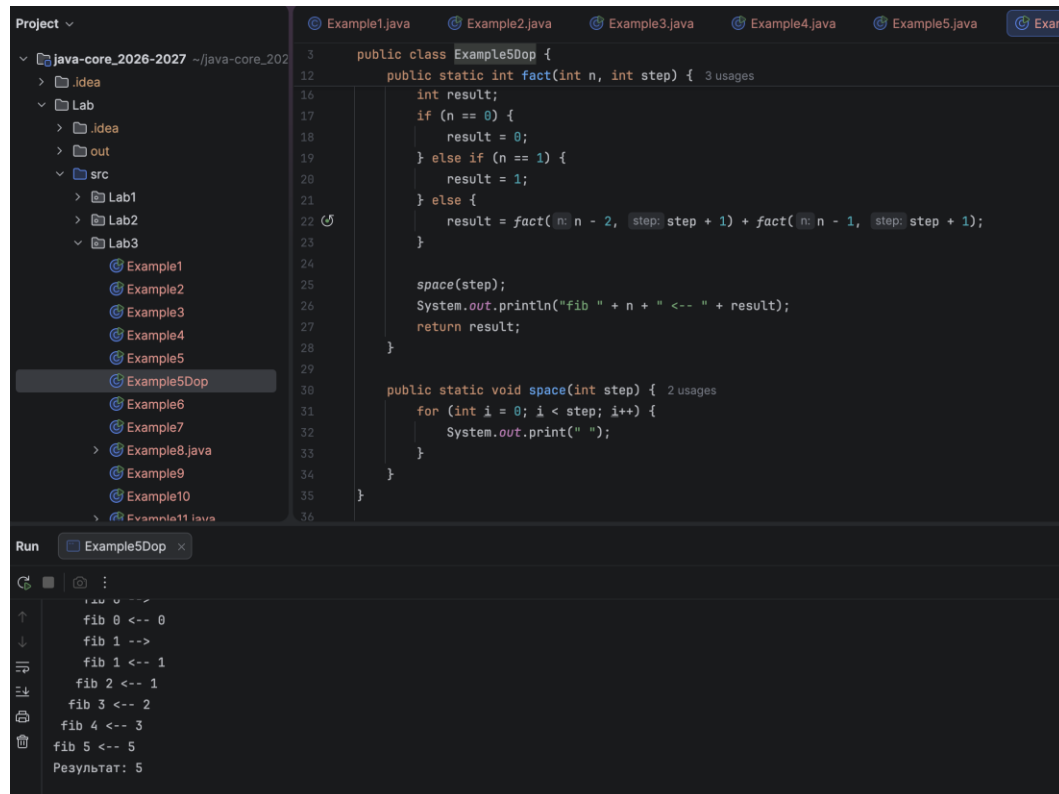
public class Example5 {

    public static void main(String[] args) { System.out.println(fact(5)); }

    public static int fact(int n) { 3 usages
        if (n == 0) {
            return 0;
        } else if (n == 1) {
            return 1;
        } else {
            return fact(n - 2) + fact(n - 1);
        }
    }
}
```

The Run window at the bottom shows the command executed: `/Library/Java/JavaVirtualMachines/temurin-17.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/Contents/lib/idea_rt.jar=59763 -Dfile.encoding=UTF-8`, and the output is `5`. The process finished with exit code 0.

Рисунок 5. Результат работы программы Example5



The screenshot shows the IntelliJ IDEA interface. On the left, the Project Explorer displays the file structure of 'java-core_2026-2027', with 'Example5Dop' selected under 'Lab3'. The main editor window shows the code for 'Example5Dop.java':

```
public class Example5Dop {
    public static int fact(int n, int step) { 3 usages
        int result;
        if (n == 0) {
            result = 0;
        } else if (n == 1) {
            result = 1;
        } else {
            result = fact(n - 2, step + 1) + fact(n - 1, step + 1);
        }

        space(step);
        System.out.println("fib " + n + " <-- " + result);
        return result;
    }

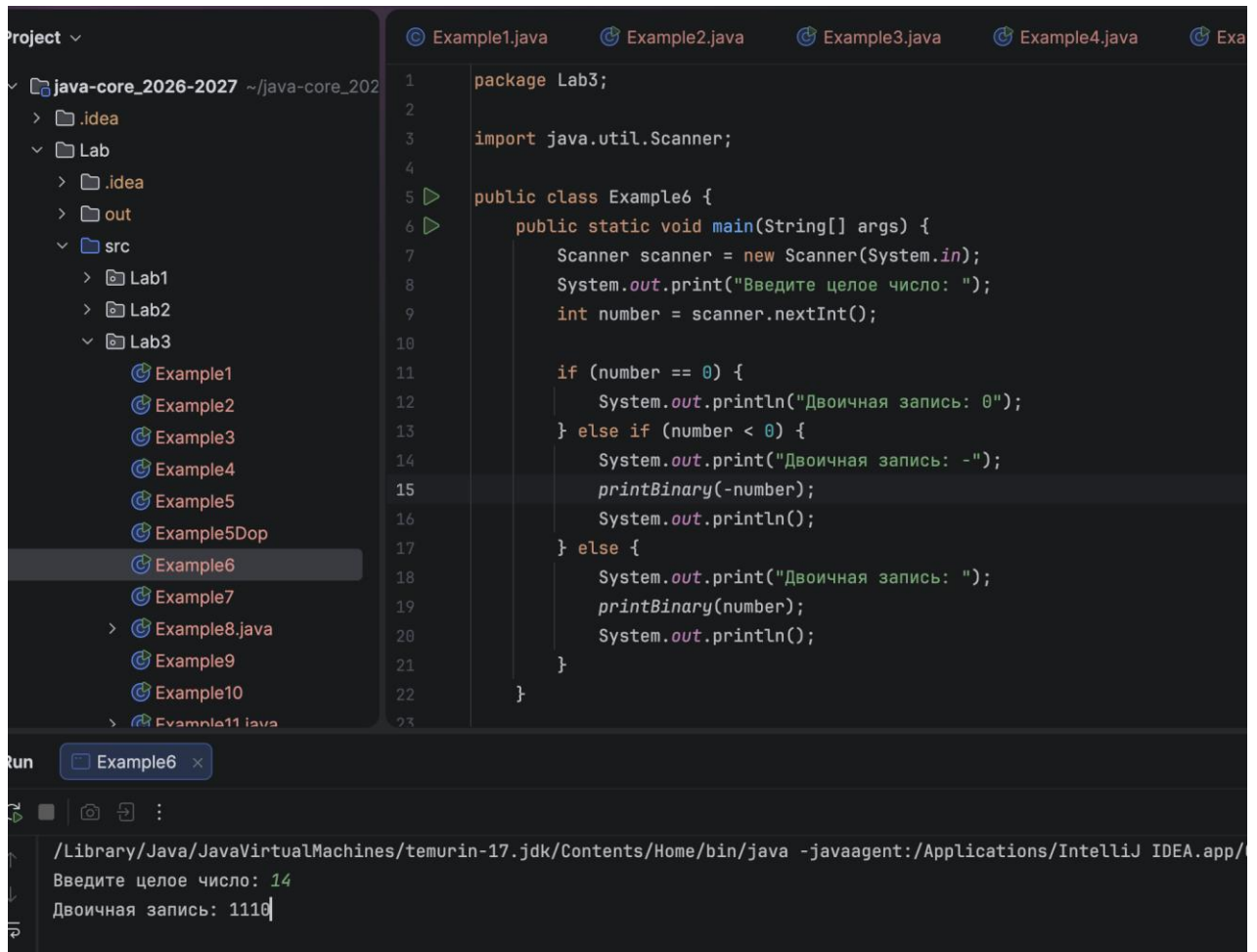
    public static void space(int step) { 2 usages
        for (int i = 0; i < step; i++) {
            System.out.print(" ");
        }
    }
}
```

The Run window at the bottom shows the command executed: `/Library/Java/JavaVirtualMachines/temurin-17.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/Contents/lib/idea_rt.jar=59763 -Dfile.encoding=UTF-8`. The output is:

```
fib 0 <-- 0
fib 1 <-- 1
fib 1 <-- 1
fib 2 <-- 1
fib 3 <-- 2
fib 4 <-- 3
fib 5 <-- 5
Результат: 5
```

Рисунок 6. Результат работы программы Example5Dop

6. Создать приложение с использованием рекурсии для перевода целого числа, введенного с клавиатуры, в двоичную систему счисления.



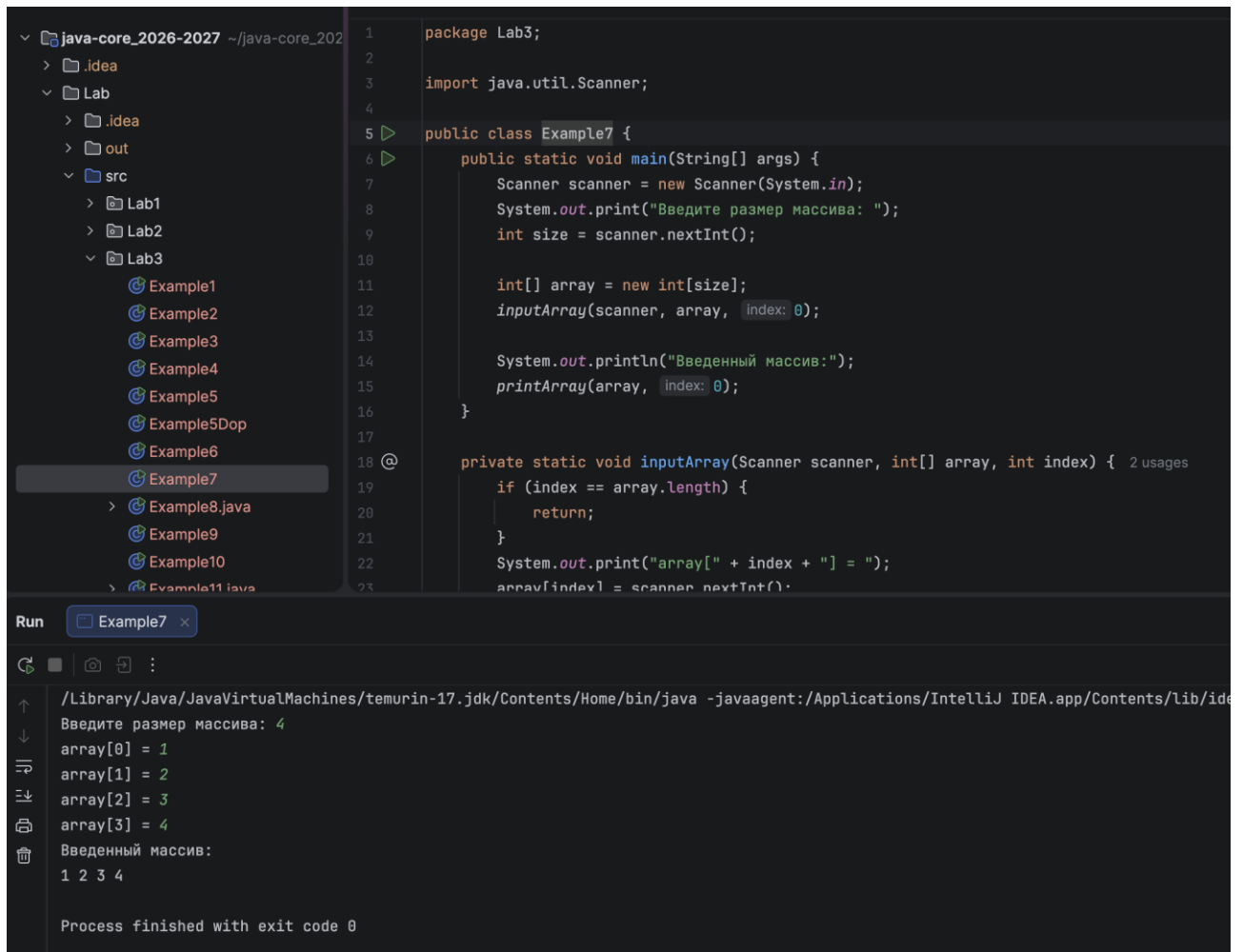
The screenshot displays the IntelliJ IDEA IDE. On the left, the Project Explorer shows a project named 'java-core_2026-2027' with a 'Lab' folder containing 'src' and 'out' subfolders. Under 'src', there are folders 'Lab1', 'Lab2', and 'Lab3'. 'Lab3' contains files 'Example1' through 'Example11.java'. 'Example6' is selected. The main editor shows the code for 'Example6.java'.

```
1 package Lab3;
2
3 import java.util.Scanner;
4
5 public class Example6 {
6     public static void main(String[] args) {
7         Scanner scanner = new Scanner(System.in);
8         System.out.print("Введите целое число: ");
9         int number = scanner.nextInt();
10
11         if (number == 0) {
12             System.out.println("Двоичная запись: 0");
13         } else if (number < 0) {
14             System.out.print("Двоичная запись: -");
15             printBinary(-number);
16             System.out.println();
17         } else {
18             System.out.print("Двоичная запись: ");
19             printBinary(number);
20             System.out.println();
21         }
22     }
23 }
```

Below the code editor, the Run window shows the execution of 'Example6'. The command line is: `/Library/Java/JavaVirtualMachines/temurin-17.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/`. The output shows the prompt 'Введите целое число: ' followed by the input '14', and then 'Двоичная запись: 1110'.

Рисунок 7. Результат работы программы Example6

7. Создать приложение, позволяющее ввести и вывести одномерный массив целых чисел. Для ввода и вывода массива разработать рекурсивные методы вместо циклов for.



The screenshot displays an IDE with a project named 'java-core_2026-2027'. The 'src' directory contains a 'Lab3' folder with several example files, including 'Example7'. The code editor shows the following Java code:

```
1 package Lab3;
2
3 import java.util.Scanner;
4
5 public class Example7 {
6     public static void main(String[] args) {
7         Scanner scanner = new Scanner(System.in);
8         System.out.print("Введите размер массива: ");
9         int size = scanner.nextInt();
10
11         int[] array = new int[size];
12         inputArray(scanner, array, index: 0);
13
14         System.out.println("Введенный массив:");
15         printArray(array, index: 0);
16     }
17
18     private static void inputArray(Scanner scanner, int[] array, int index) { 2 usages
19         if (index == array.length) {
20             return;
21         }
22         System.out.print("array[" + index + "] = ");
23         array[index] = scanner.nextInt();
```

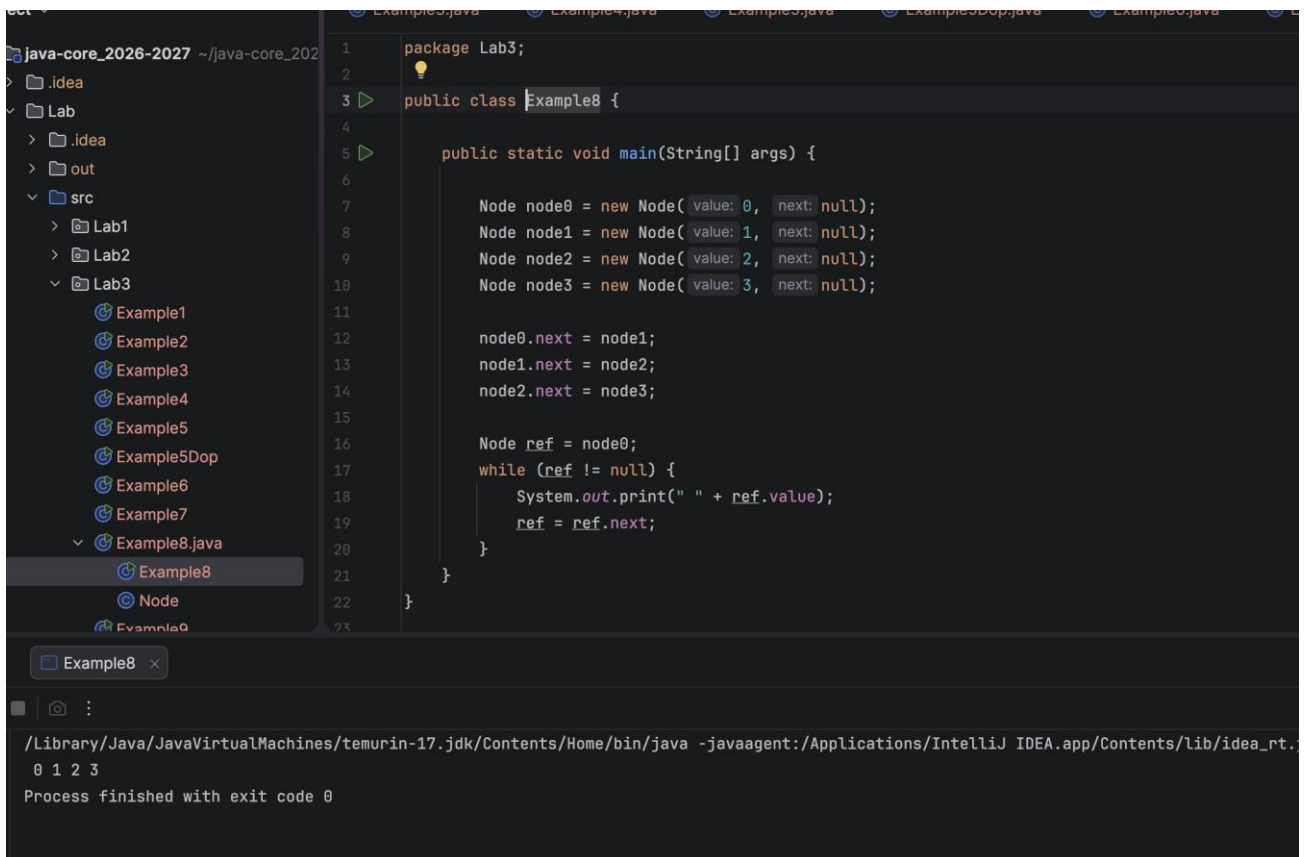
The 'Run' window at the bottom shows the execution of 'Example7'. The output is as follows:

```
/Library/Java/JavaVirtualMachines/temurin-17.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/Contents/lib/ide
Введите размер массива: 4
array[0] = 1
array[1] = 2
array[2] = 3
array[3] = 4
Введенный массив:
1 2 3 4

Process finished with exit code 0
```

Рисунок 8. Результат работы программы Example7

8. Выполнить пример 1 из раздела 2: построить однонаправленный список из предварительно созданных независимых элементов и вывести значения полей на экран.



The screenshot displays the IntelliJ IDEA IDE. On the left, the Project Explorer shows a directory structure with 'Lab3' containing several example files, with 'Example8' selected. The main editor window shows the source code for 'Example8.java' in the 'Lab3' package. The code defines a 'Node' class and a 'main' method that creates a linked list with four nodes (values 0, 1, 2, 3) and prints their values sequentially. The bottom panel shows the Run output, which displays the values '0 1 2 3' and confirms the process finished with exit code 0.

```
1 package Lab3;
2
3 public class Example8 {
4
5     public static void main(String[] args) {
6
7         Node node0 = new Node( value: 0, next: null);
8         Node node1 = new Node( value: 1, next: null);
9         Node node2 = new Node( value: 2, next: null);
10        Node node3 = new Node( value: 3, next: null);
11
12        node0.next = node1;
13        node1.next = node2;
14        node2.next = node3;
15
16        Node ref = node0;
17        while (ref != null) {
18            System.out.print(" " + ref.value);
19            ref = ref.next;
20        }
21    }
22 }
23 }
```

Example8 x

/Library/Java/JavaVirtualMachines/temurin-17.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/Contents/lib/idea_rt. 0 1 2 3
Process finished with exit code 0

Рисунок 9. Результат работы программы Example8

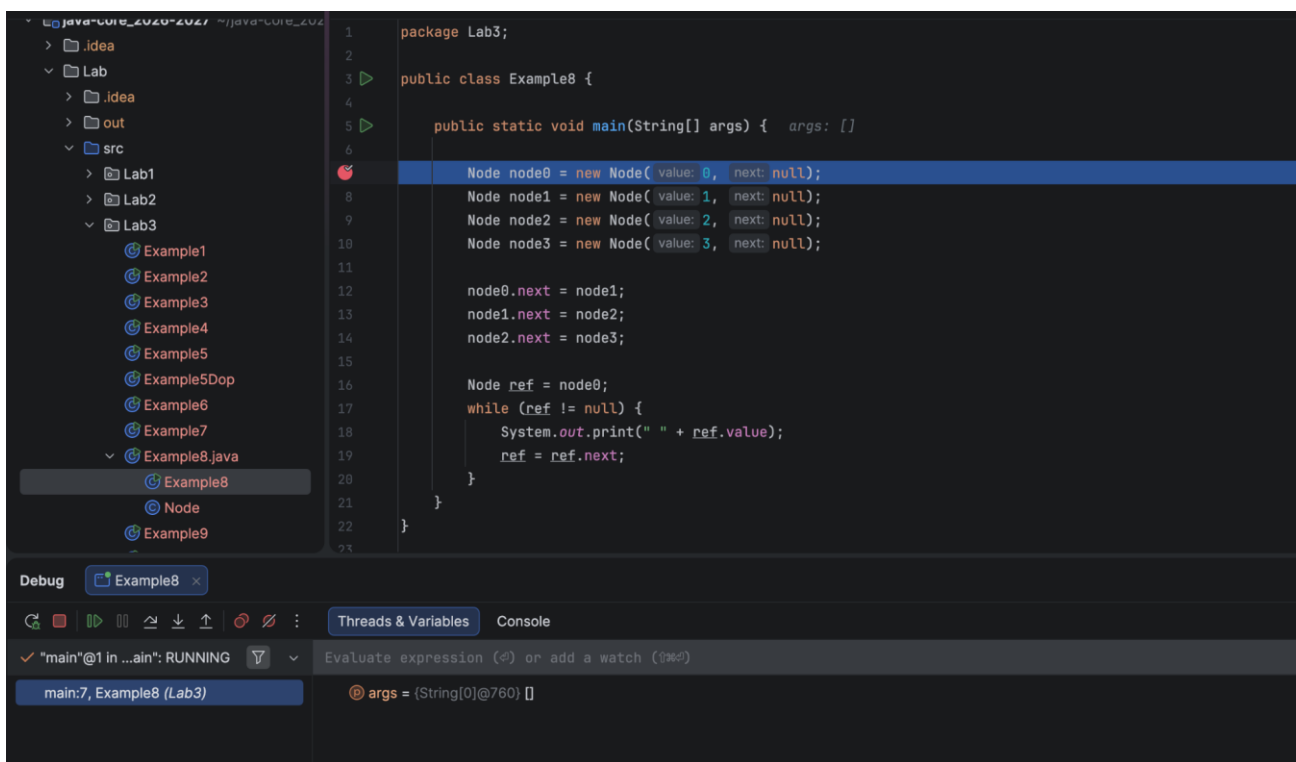


Рисунок 10. Первая точка отладки программы Example8

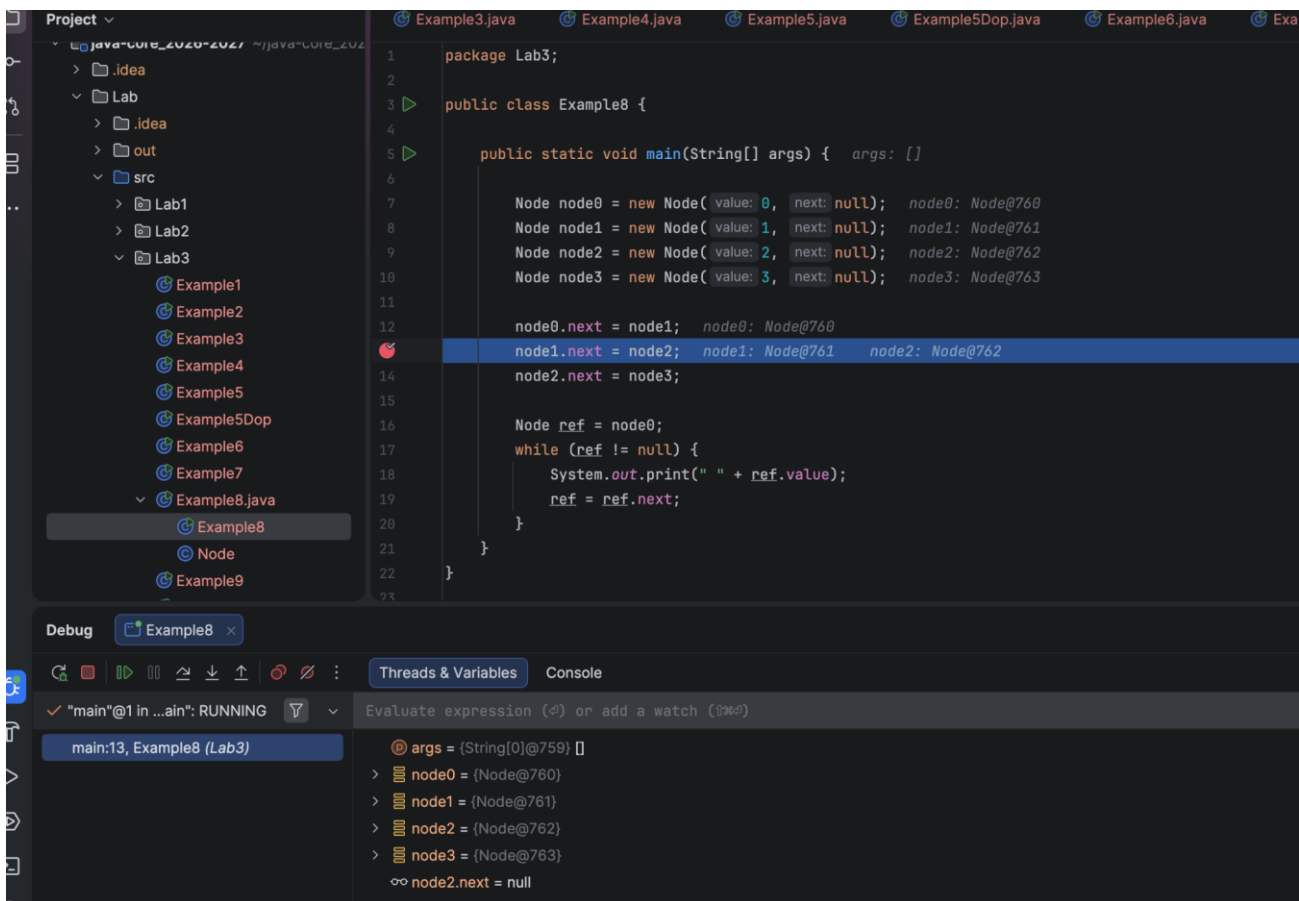


Рисунок 11. Вторая точка отладки программы Example8

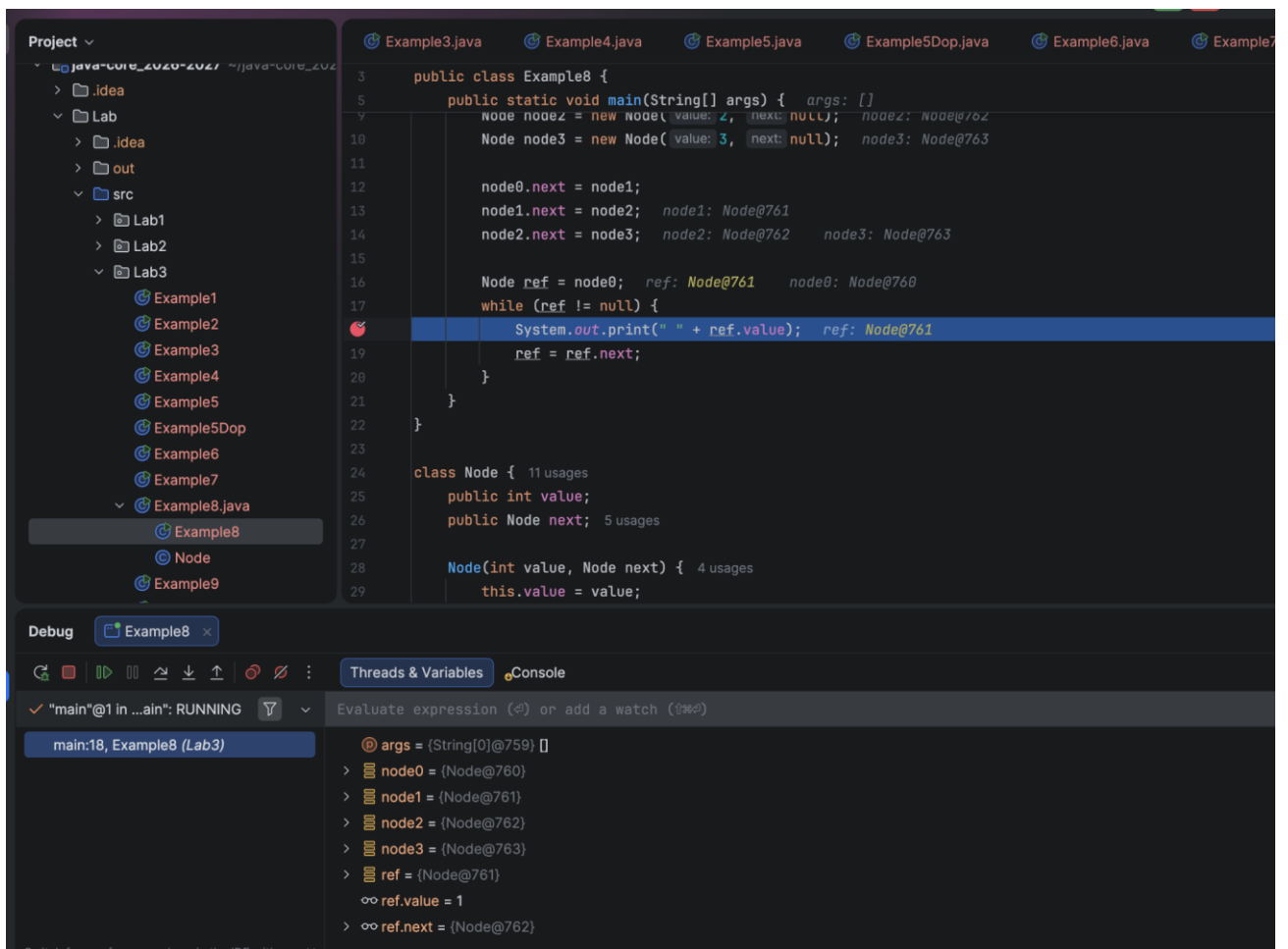
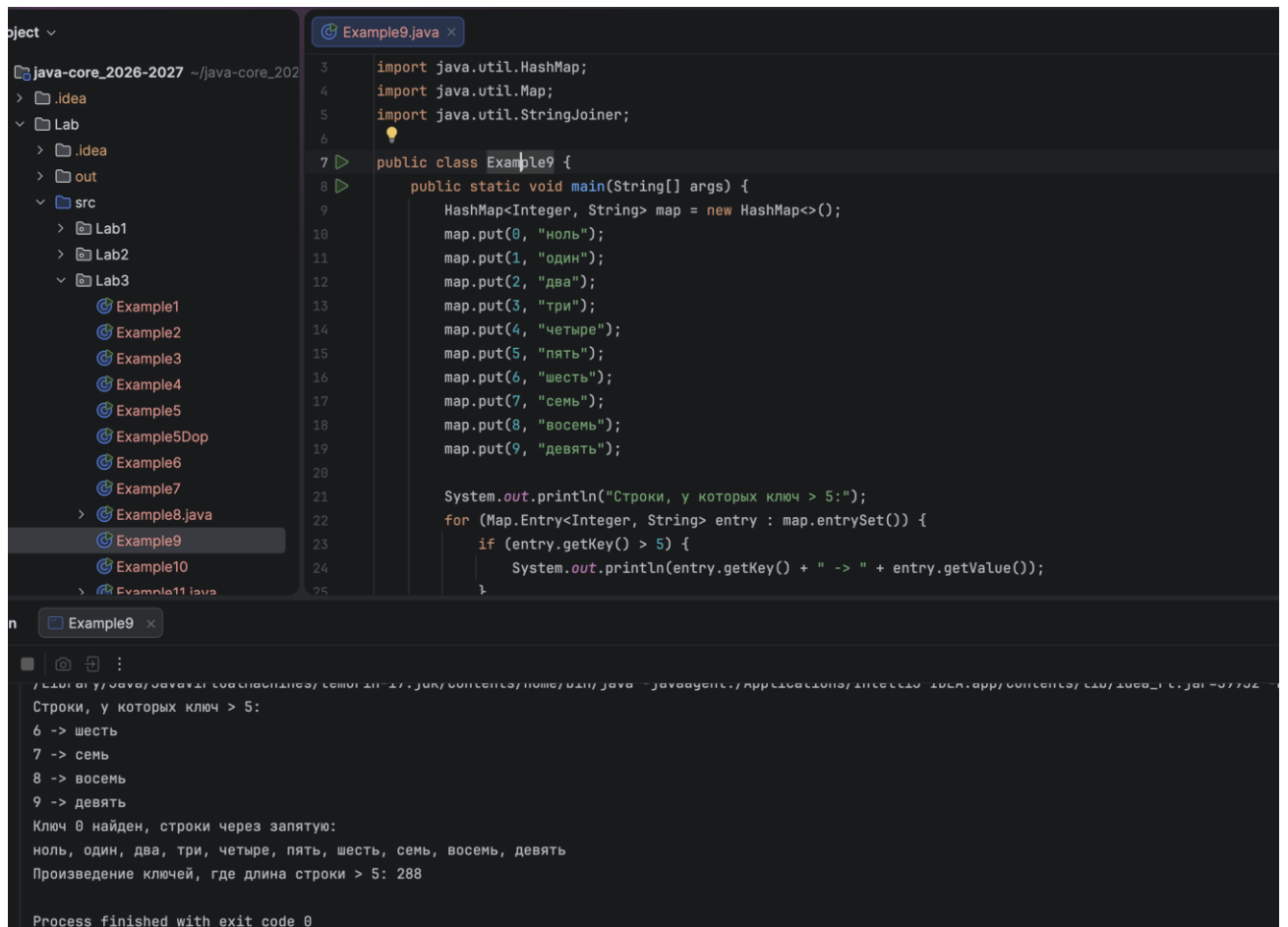


Рисунок 12. Третья точка отладки программы Example8

9. Заполнить HashMap 10 объектами <Integer, String>. Найти строки, у которых ключ > 5. Если ключ = 0, вывести строки через запятую. Перемножить все ключи, где длина строки > 5.



```
import java.util.HashMap;
import java.util.Map;
import java.util.StringJoiner;

public class Example9 {
    public static void main(String[] args) {
        HashMap<Integer, String> map = new HashMap<>();
        map.put(0, "ноль");
        map.put(1, "один");
        map.put(2, "два");
        map.put(3, "три");
        map.put(4, "четыре");
        map.put(5, "пять");
        map.put(6, "шесть");
        map.put(7, "семь");
        map.put(8, "восемь");
        map.put(9, "девять");

        System.out.println("Строки, у которых ключ > 5:");
        for (Map.Entry<Integer, String> entry : map.entrySet()) {
            if (entry.getKey() > 5) {
                System.out.println(entry.getKey() + " -> " + entry.getValue());
            }
        }

        System.out.println("Ключ 0 найден, строки через запятую:");
        StringJoiner joiner = new StringJoiner(", ");
        for (Map.Entry<Integer, String> entry : map.entrySet()) {
            if (entry.getKey() == 0) {
                joiner.add(entry.getValue());
            }
        }
        System.out.println(joiner.toString());

        int product = 1;
        for (Map.Entry<Integer, String> entry : map.entrySet()) {
            if (entry.getValue().length() > 5) {
                product *= entry.getKey();
            }
        }
        System.out.println("Произведение ключей, где длина строки > 5: " + product);
    }
}
```

Строки, у которых ключ > 5:

6 -> шесть
7 -> семь
8 -> восемь
9 -> девять

Ключ 0 найден, строки через запятую:

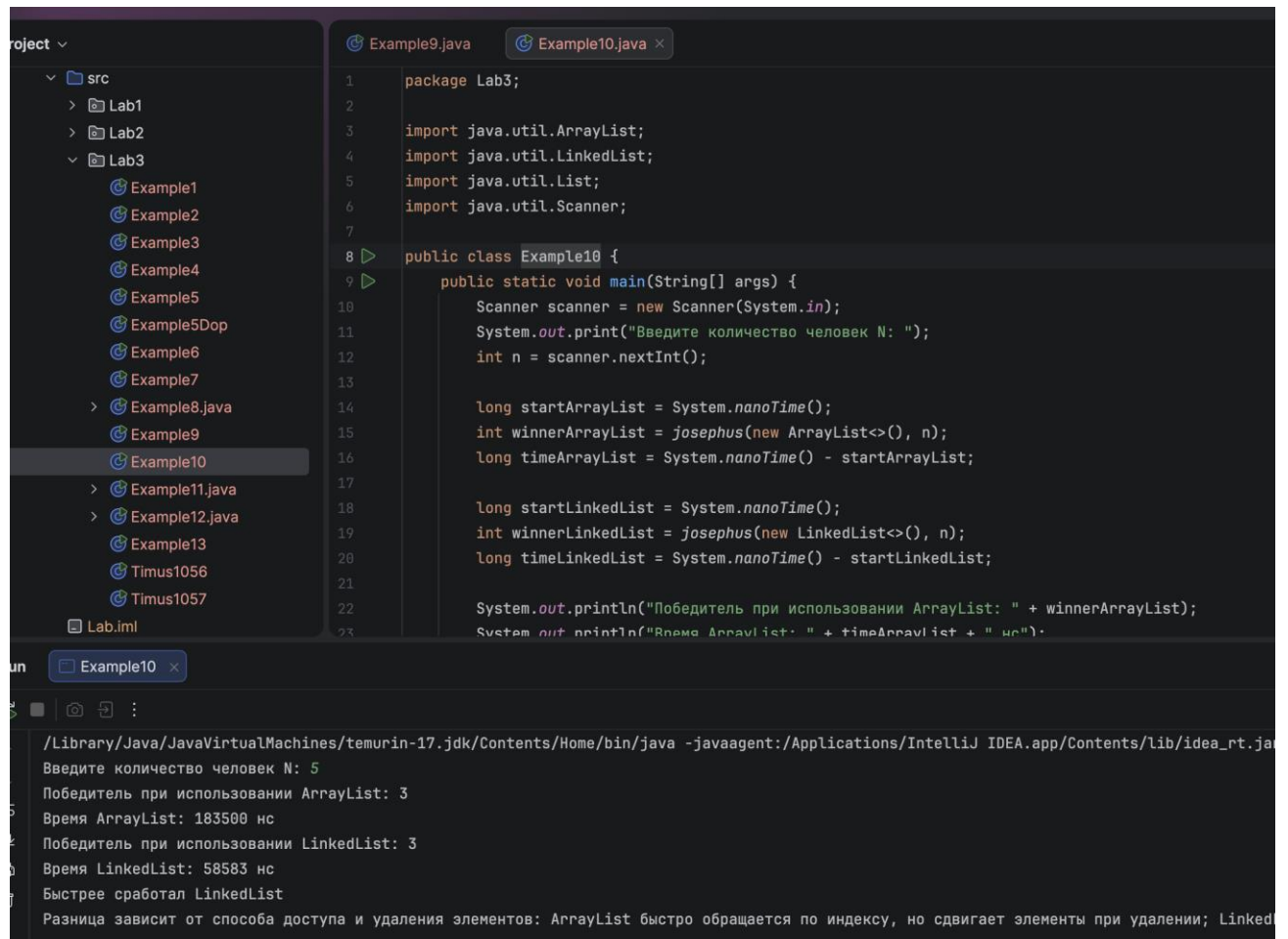
ноль, один, два, три, четыре, пять, шесть, семь, восемь, девять

Произведение ключей, где длина строки > 5: 288

Process finished with exit code 0

Рисунок 13. Результат работы программы Example9

10. В кругу стоят N человек, пронумерованных от 1 до N . При ведении счета по кругу вычеркивается каждый второй человек, пока не останется один. Составить две реализации: с использованием `ArrayList` и с использованием `LinkedList`. Сравнить время выполнения.



```
1 package Lab3;
2
3 import java.util.ArrayList;
4 import java.util.LinkedList;
5 import java.util.List;
6 import java.util.Scanner;
7
8 public class Example10 {
9     public static void main(String[] args) {
10         Scanner scanner = new Scanner(System.in);
11         System.out.print("Введите количество человек N: ");
12         int n = scanner.nextInt();
13
14         long startArrayList = System.nanoTime();
15         int winnerArrayList = josephus(new ArrayList<>(), n);
16         long timeArrayList = System.nanoTime() - startArrayList;
17
18         long startLinkedList = System.nanoTime();
19         int winnerLinkedList = josephus(new LinkedList<>(), n);
20         long timeLinkedList = System.nanoTime() - startLinkedList;
21
22         System.out.println("Победитель при использовании ArrayList: " + winnerArrayList);
23         System.out.println("Время ArrayList: " + timeArrayList + " нс");
24     }
25
26     private static int josephus(List<Integer> list, int n) {
27         if (list.size() == 1) return list.get(0);
28         int index = 0;
29         while (index != n - 1) {
30             index = (index + 1) % list.size();
31         }
32         list.remove(index);
33         return josephus(list, n);
34     }
35 }
```

```
/Library/Java/JavaVirtualMachines/temurin-17.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/Contents/lib/idea_rt.jar
Введите количество человек N: 5
Победитель при использовании ArrayList: 3
Время ArrayList: 183500 нс
Победитель при использовании LinkedList: 3
Время LinkedList: 58583 нс
Быстрее сработал LinkedList
Разница зависит от способа доступа и удаления элементов: ArrayList быстро обращается по индексу, но сдвигает элементы при удалении; LinkedList
```

Рисунок 14. Результат работы программы Example10

11. Продемонстрировать два способа создания линейного однонаправленного списка: с головы и с хвоста. Отработать код с помощью отладчика.

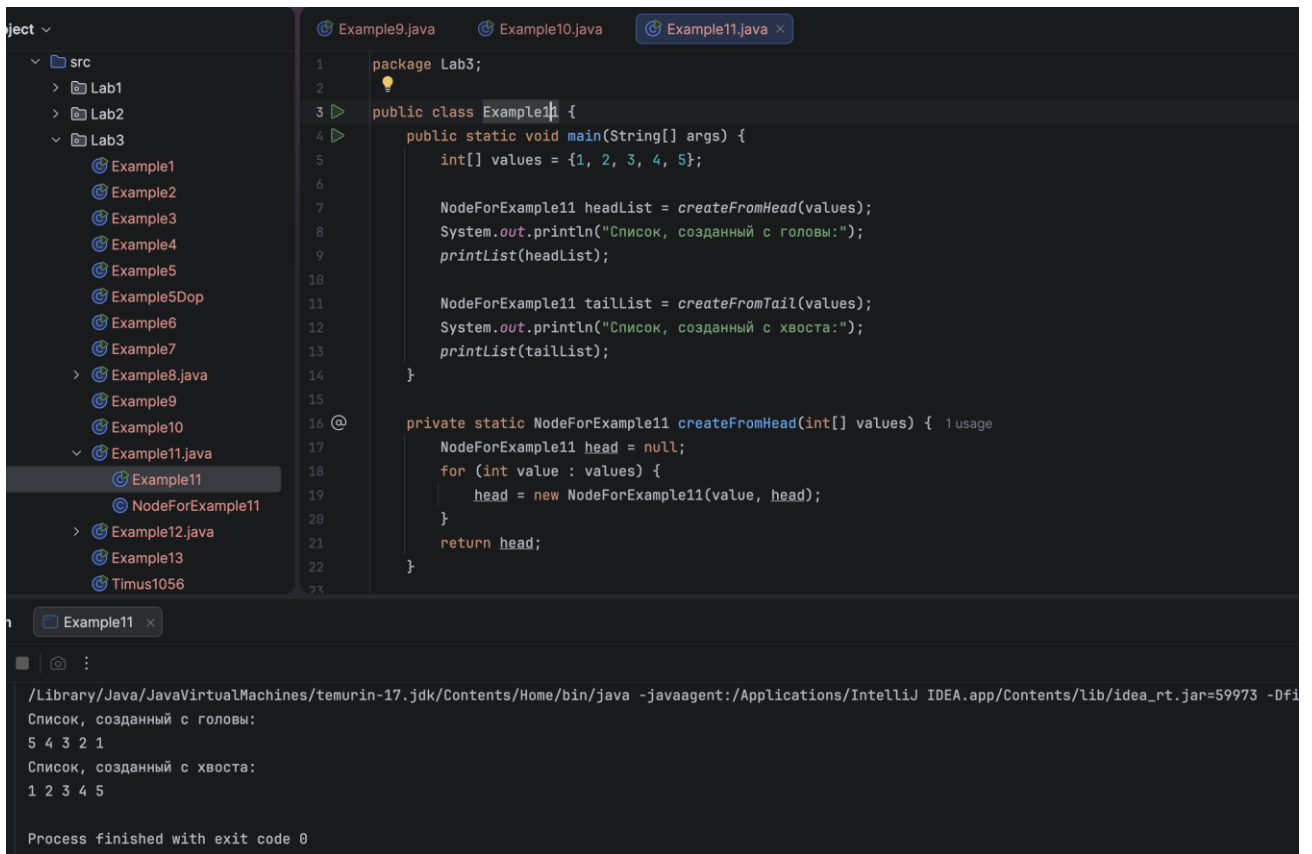


Рисунок 15. Результат работы программы Example11

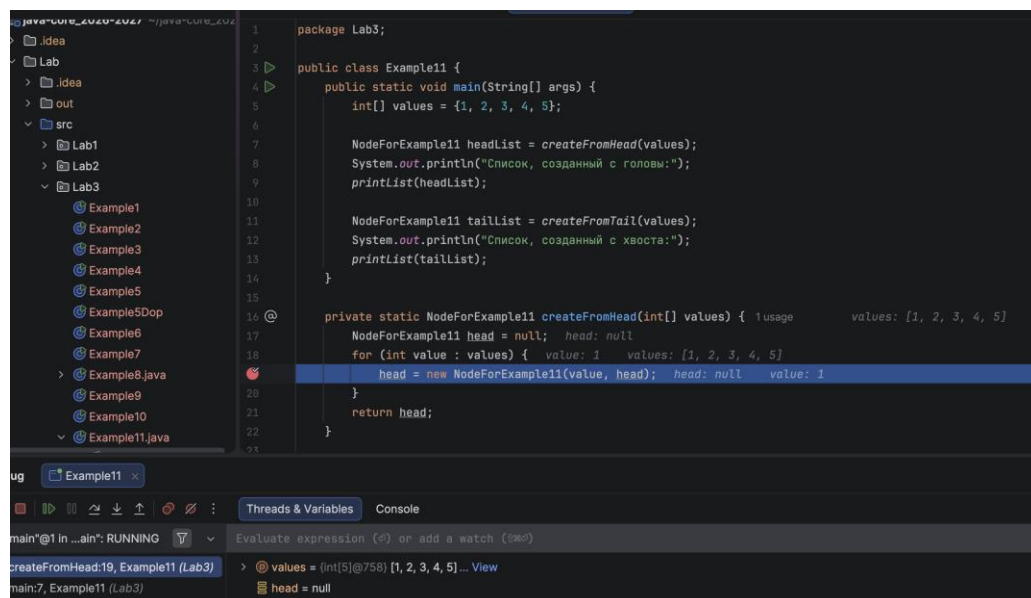


Рисунок 16. Первая точка отладки программы Example11

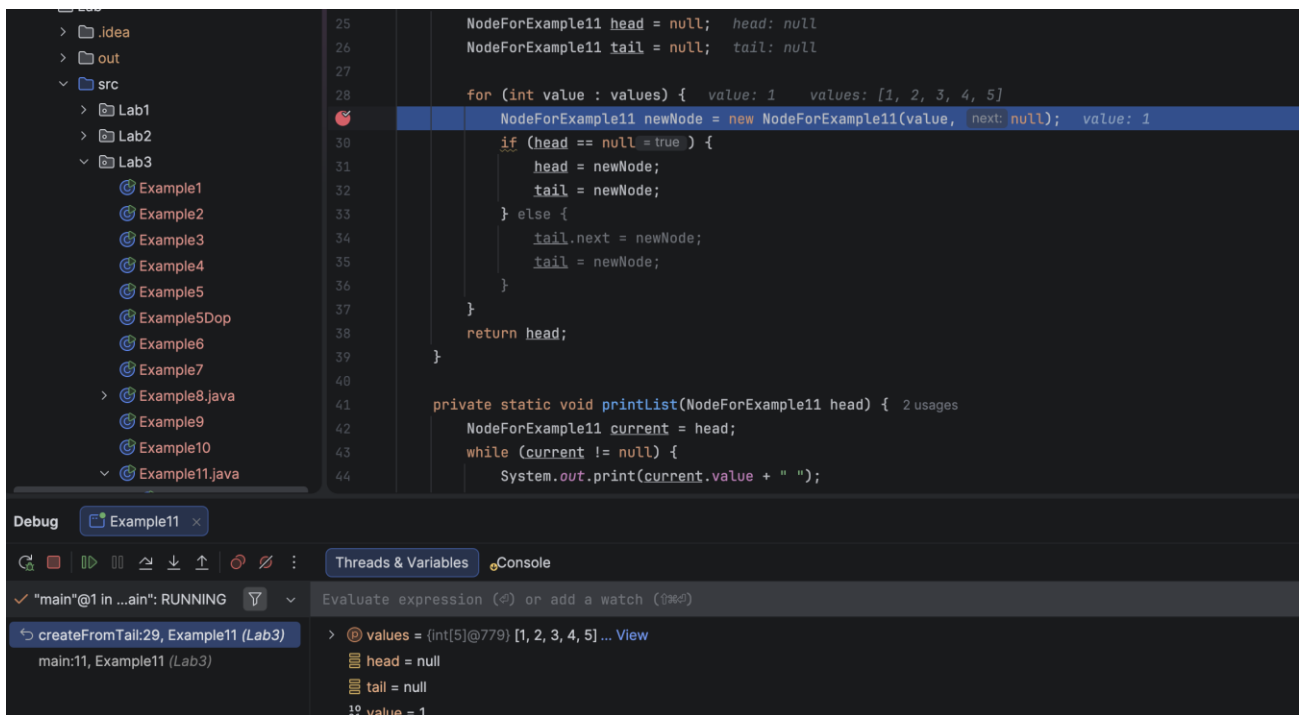


Рисунок 17. Вторая точка отладки программы Example11

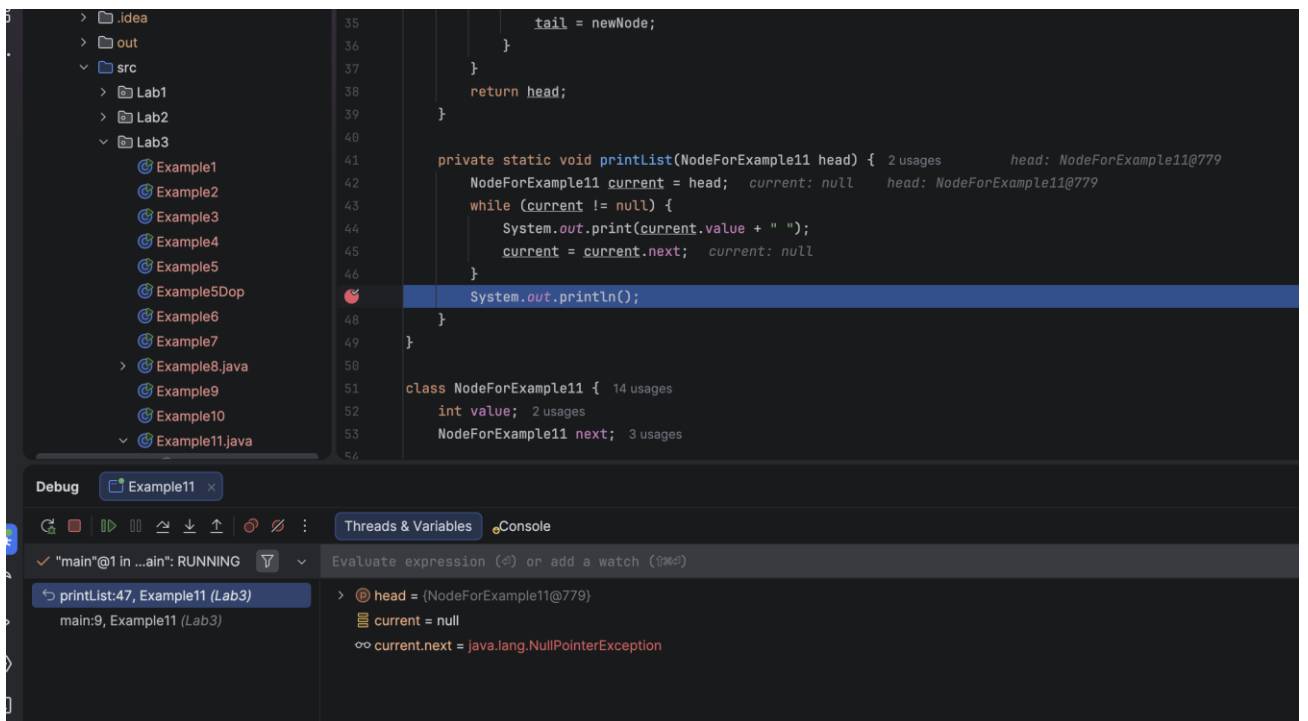


Рисунок 18. Третья точка отладки программы Example11

12. Разработать проект, в котором для ввода, вывода и изменения односвязного линейного списка созданы методы `createHead()`, `createTail()`, `toString()`, `addFirst()`, `addLast()`, `insert()`, `removeFirst()`, `removeLast()`, `remove()`, `createHeadRec()`, `createTailRec()`, `toStringRec()`.

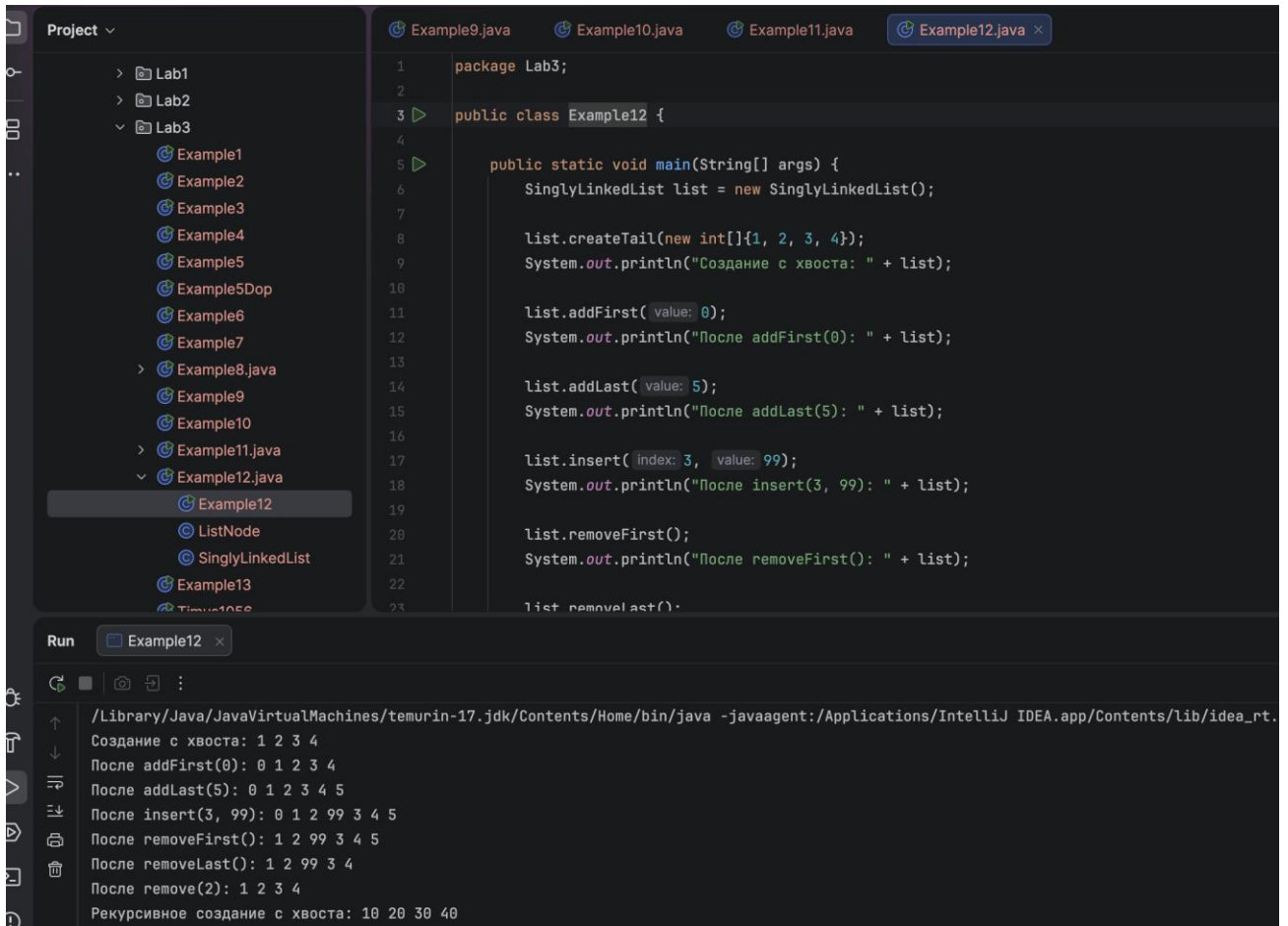
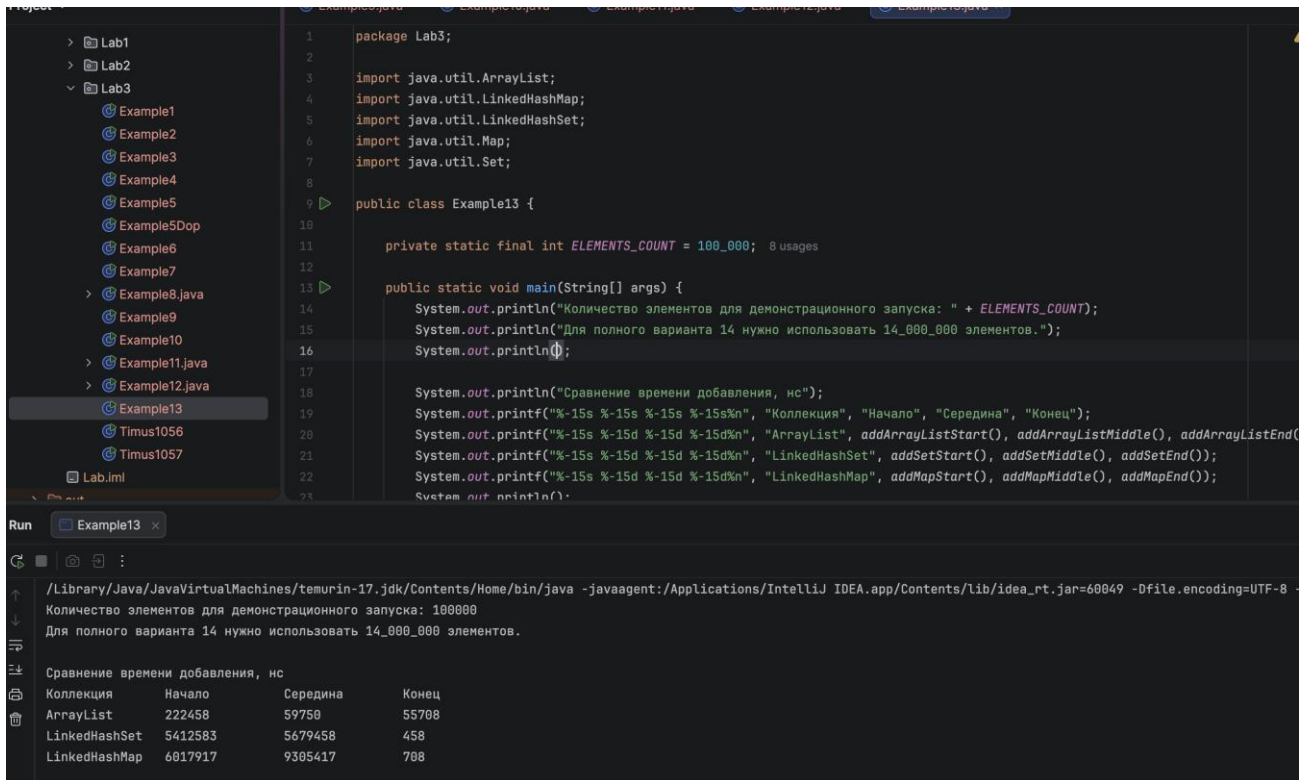


Рисунок 19. Результат работы программы Example12

13. Выполнить сравнение коллекций по варианту 14: ArrayList, LinkedHashSet, LinkedHashMap. Измерить время добавления, удаления и получения элемента.



The screenshot displays the source code of `Example13.java` in an IDE. The code defines a class `Example13` with a `main` method that compares the performance of `ArrayList`, `LinkedHashSet`, and `LinkedHashMap` for adding, deleting, and retrieving elements. The `main` method uses `System.out.printf` to output the results in a table format.

```
1 package Lab3;
2
3 import java.util.ArrayList;
4 import java.util.LinkedHashMap;
5 import java.util.LinkedHashSet;
6 import java.util.Map;
7 import java.util.Set;
8
9 public class Example13 {
10
11     private static final int ELEMENTS_COUNT = 100_000; 8 usages
12
13     public static void main(String[] args) {
14         System.out.println("Количество элементов для демонстрационного запуска: " + ELEMENTS_COUNT);
15         System.out.println("Для полного варианта 14 нужно использовать 14_000_000 элементов.");
16         System.out.println();
17
18         System.out.println("Сравнение времени добавления, нс");
19         System.out.printf("%-15s %-15s %-15s %-15s\n", "Коллекция", "Начало", "Середина", "Конец");
20         System.out.printf("%-15s %-15d %-15d %-15d\n", "ArrayList", addArrayListStart(), addArrayListMiddle(), addArrayListEnd());
21         System.out.printf("%-15s %-15d %-15d %-15d\n", "LinkedHashSet", addSetStart(), addSetMiddle(), addSetEnd());
22         System.out.printf("%-15s %-15d %-15d %-15d\n", "LinkedHashMap", addMapStart(), addMapMiddle(), addMapEnd());
23         System.out.println();
24     }
25 }
```

The Run window shows the output of the program, which includes the number of elements for the demonstration (100,000) and a table comparing the performance of the three collections.

Коллекция	Начало	Середина	Конец
ArrayList	222458	59750	55708
LinkedHashSet	5412583	5679458	458
LinkedHashMap	6017917	9305417	708

Рисунок 20. Результат работы программы Example13

Общее описание коллекций в Java

Интерфейс Collection является базовым интерфейсом для большинства коллекций Java, которые хранят группы объектов. На его основе работают списки, множества и очереди. Он определяет общие операции: добавление, удаление, проверку наличия элемента, получение размера и перебор элементов.

Интерфейс Map не наследуется от Collection, потому что хранит данные в виде пар «ключ – значение», а не как простую последовательность отдельных элементов. Основные операции Map связаны с добавлением, удалением и получением значения по ключу.

Схема работы коллекций Java

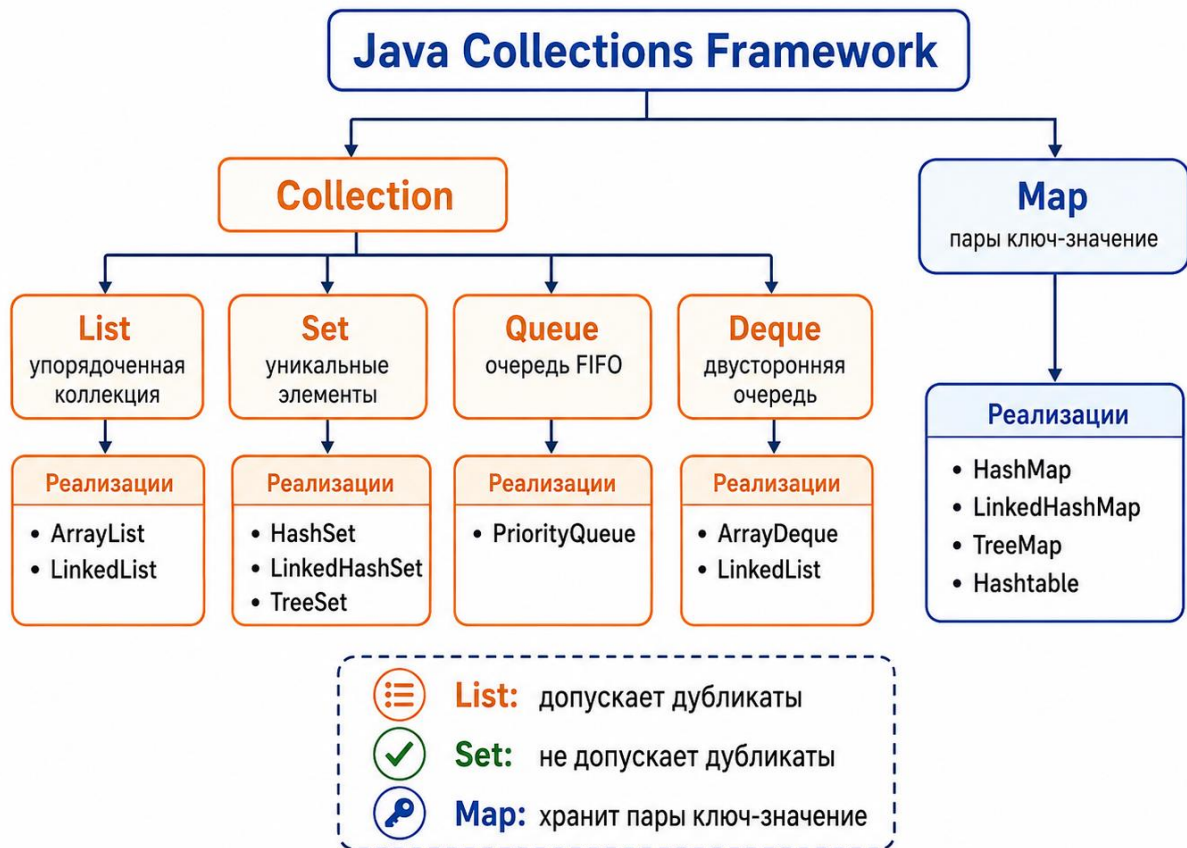


Рисунок 21. Иерархия интерфейса Collection

Иерархия интерфейса Map

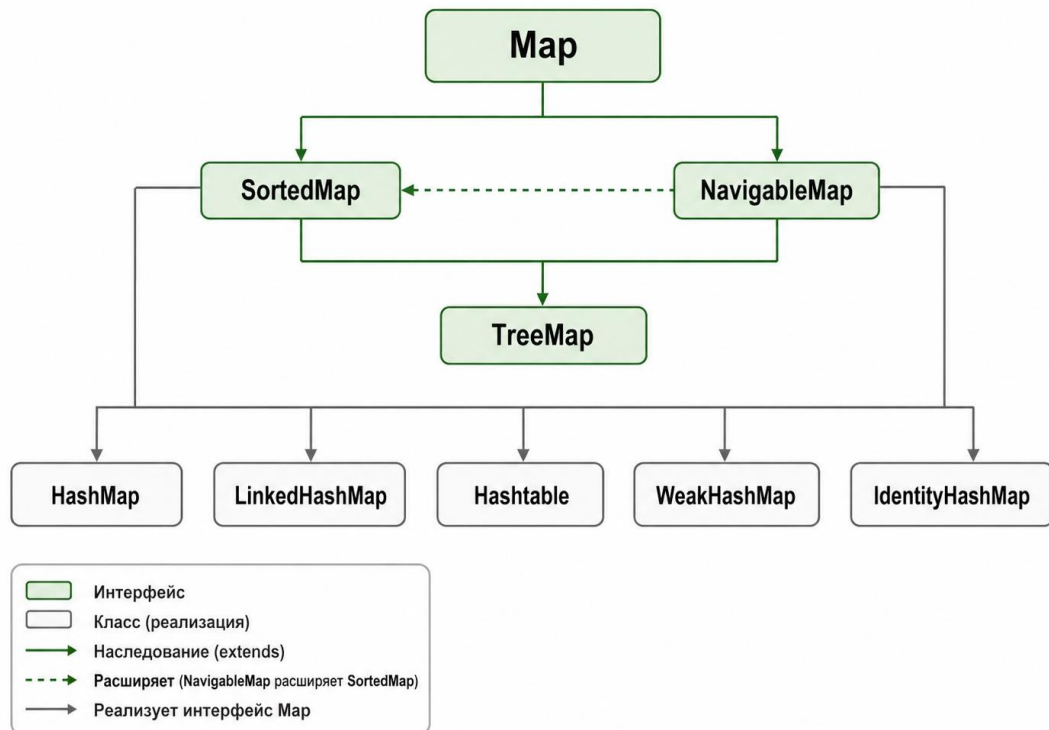


Рисунок 22. Иерархия интерфейса Map

Иерархия интерфейса Collection

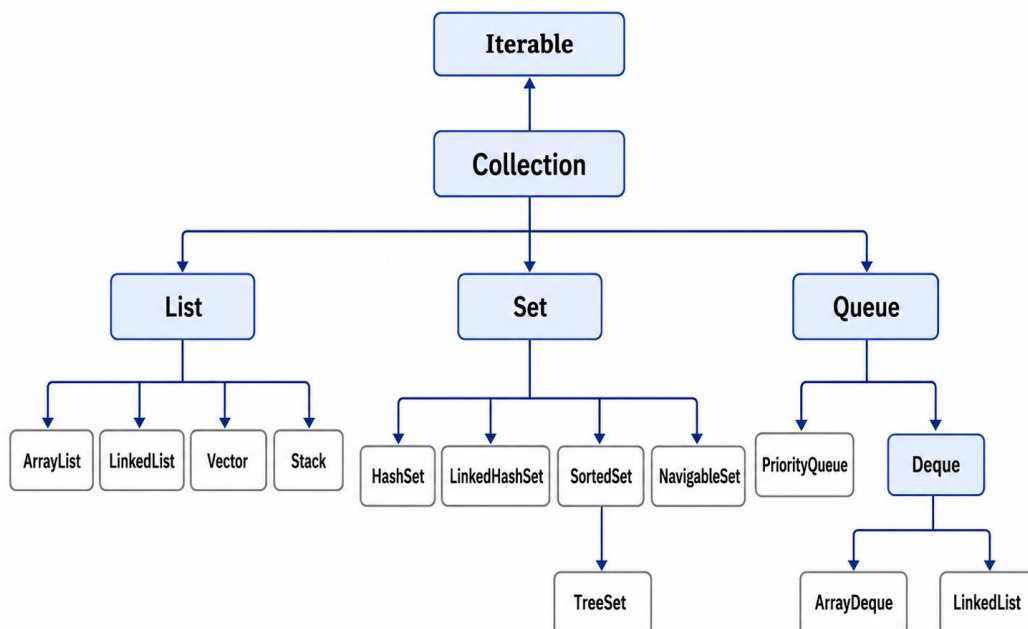


Рисунок 23. Схема работы коллекций Java

Таблица 1. Основные методы интерфейса Collection

Метод	Назначение
add(E e)	Добавляет элемент в коллекцию.
remove(Object o)	Удаляет указанный элемент из коллекции.
contains(Object o)	Проверяет наличие элемента.
size()	Возвращает количество элементов.
isEmpty()	Проверяет, пуста ли коллекция.
clear()	Удаляет все элементы.
iterator()	Возвращает итератор для обхода коллекции.

Таблица 2. Основные методы интерфейса Map

Метод	Назначение
put(K key, V value)	Добавляет пару «ключ – значение» или заменяет значение по существующему ключу.
get(Object key)	Возвращает значение по ключу.
remove(Object key)	Удаляет запись по ключу.
containsKey(Object key)	Проверяет наличие ключа.
containsValue(Object value)	Проверяет наличие значения.
keySet()	Возвращает множество ключей.
entrySet()	Возвращает набор пар «ключ – значение».

Описание коллекций варианта 14

ArrayList

ArrayList реализован на основе динамического массива. Элементы хранятся упорядоченно, в порядке добавления. Получение элемента по индексу выполняется быстро, так как используется прямой доступ к ячейке массива. При добавлении или удалении элемента в начале или середине коллекции элементы сдвигаются, поэтому такие операции выполняются медленнее.

Сложность: получение по индексу - $O(1)$; добавление в конец в среднем - $O(1)$; добавление или удаление в начале/середине - $O(n)$; поиск по значению - $O(n)$.

LinkedHashSet

LinkedHashSet хранит только уникальные элементы и сохраняет порядок их вставки. Внутри используется хеш-таблица и связный список, который поддерживает порядок обхода. Получения по индексу нет, поэтому для доступа к элементу по позиции приходится выполнять перебор.

Сложность: добавление, удаление и поиск по значению в среднем - $O(1)$; перебор до нужной позиции - $O(n)$. Порядок элементов сохраняется в порядке добавления.

LinkedHashMap

LinkedHashMap хранит пары «ключ – значение» и сохраняет порядок вставки записей. Внутри используется хеш-таблица для быстрого доступа по ключу и связный список для сохранения порядка обхода. При добавлении записи с уже существующим ключом старое значение заменяется новым.

Сложность: добавление, удаление и получение по ключу в среднем - $O(1)$; перебор до позиции по порядку вставки - $O(n)$. Элементы хранятся как пары ключей и значений.

Сравнение коллекций

Для варианта 14 количество элементов в коллекции по условию равно $14 * 1\,000\,000 = 14\,000\,000$. В демонстрационном запуске в программе можно уменьшить значение константы `ELEMENTS_COUNT`, чтобы код выполнялся быстрее на обычном компьютере.

```
/Library/Java/JavaVirtualMachines/temurin-17.jdk/Contents/Home/bin/java -javaagent:/Applicatio
Количество элементов для демонстрационного запуска: 100000
Для полного варианта 14 нужно использовать 14_000_000 элементов.

Сравнение времени добавления, нс
Коллекция      Начало      Середина      Конец
ArrayList      222458      59750         55708
LinkedHashSet  5412583    5679458       458
LinkedHashMap  6017917    9305417       708

Сравнение времени удаления, нс
Коллекция      Начало      Середина      Конец
ArrayList      10084       4167          375
LinkedHashSet  2500        1875          3334
LinkedHashMap  2458        792           833

Сравнение времени получения элемента, нс
Коллекция      Способ доступа      Время
ArrayList      get(index)           2167
LinkedHashSet  итерация до середины 128542
LinkedHashMap  get(key)             18833

Process finished with exit code 0
```

Рисунок 24. Результат сравнения коллекций варианта 14

Ответы на вопросы

Вопрос №14. Почему в классе `Iterator` нет метода для получения следующего элемента без передвижения курсора?

`Iterator` предназначен для последовательного однонаправленного обхода коллекции. Его основной метод `next()` одновременно возвращает следующий элемент и передвигает внутренний курсор. Отдельного метода для просмотра следующего элемента без сдвига курсора нет, потому что не каждая коллекция может эффективно поддерживать такую операцию. Для универсального интерфейса `Iterator` оставлены только базовые действия: `hasNext()`, `next()` и `remove()`.

Если требуется посмотреть следующий элемент без продвижения, обычно используют дополнительную переменную, собственную обертку над итератором или другие структуры данных. Например, у `Queue` есть метод `peek()`, который возвращает элемент без удаления, но это уже другой интерфейс и другая логика работы.

Задачи Timus

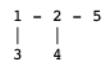
1056. Центры сети

Ограничение времени: 2.0 секунды

Ограничение памяти: 64 МБ

Вступление

Компьютерная сеть была создана путем последовательного подключения компьютеров к тем компьютерам, которые уже были подключены к сети. Каждый новый компьютер в сети получает порядковый номер, а в протокол сети дописывается порядковый номер его родительского компьютера. Таким образом, протокол сети – это последовательность целых чисел: первое из них всегда 1, потому что второй компьютер может быть подключен только к первому, второе число – 1 или 2, и так далее. Общее количество чисел в протоколе равно $N - 1$ (N – общее количество компьютеров). Например, протокол 1, 1, 2, 2 соответствует следующей сети:



Расстояние между компьютерами – это количество взаимных соединений на пути между этими компьютерами. Так, в приведенном выше примере расстояние между компьютерами №4 и №5 равно 2, а между №3 и №5 равно 3.

Определение. Центром сети называется компьютер, имеющий минимальное расстояние до самого удаленного от него компьютера. В приведенном примере компьютеры №1 и №2 являются центрами сети.

Задача

Найдите все центры сети, зная ее протокол.

Исходные данные

В первой строке содержится целое число N – количество компьютеров в сети ($2 \leq N \leq 10000$). Далее $N - 1$ строка содержит протокол сети.

Результат

Выведите порядковые номера всех центров сети в порядке возрастания.

Пример

исходные данные	результат
5 1 1 2 2	1 2

Источник задачи: Rybinsk State Avia Academy

Метки: [графы](#) ([скрыть метки для нерешенных задач](#))

Сложность: 550 [Версия для печати](#) [Отправить на проверку](#) [Обсуждение на форуме \(35\)](#)

[Все попытки \(10635\)](#) [Все успешные попытки \(3697\)](#) [Рейтинг решений \(2774\)](#)

Рисунок 25. Дано задачи 1056

```
1 package Lab3;
2
3 import ...
4
5
6 public class Timus1056 {
7
8     private static List<Integer>[] graph; 6 usages
9
10    public static void main(String[] args) throws Exception {
11        FastScanner scanner = new FastScanner(System.in);
12
13        int n = scanner.nextInt();
14        graph = new ArrayList[n + 1];
15
16        for (int i = 1; i <= n; i++) {
17            graph[i] = new ArrayList<>();
18        }
19
20        for (int i = 2; i <= n; i++) {
21            int parent = scanner.nextInt();
22            graph[i].add(parent);
23            graph[parent].add(i);
24        }
25    }
26 }
```

Timus1056

```
/Library/Java/JavaVirtualMachines/temurin-17.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/Contents/lib/idea_rt.jar=68
5
1
1
2
2
1 2
Process finished with exit code 0
```

Рисунок 26. Решение задачи 1056

1057. Количество степеней

Ограничение времени: 1.0 секунды

Ограничение памяти: 64 МБ

Напишите программу для определения количества целых чисел, лежащих на отрезке $[X; Y]$ и представляющих собой сумму ровно K различных степеней целого числа B с целыми показателями.

Пример. Пусть $X = 15$, $Y = 20$, $K = 2$, $B = 2$. В данном примере три целых числа представляют собой сумму ровно двух целых степеней числа 2:

$$17 = 2^4 + 2^0,$$

$$18 = 2^4 + 2^1,$$

$$20 = 2^4 + 2^2.$$

Исходные данные

Первая строка содержит целые числа X и Y ($1 \leq X \leq Y \leq 2^{31} - 1$). Следующие две строки содержат целые числа K и B ($1 \leq K \leq 20$; $2 \leq B \leq 10$).

Результат

Выведите количество целых чисел, лежащих между X и Y , которые представляют собой сумму ровно K различных степеней целого числа B с целыми показателями.

Пример

исходные данные	результат
15 20 2 2	3

Источник задачи: Rybinsk State Avia Academy

Метки: нет ([скрыть метки для нерешенных задач](#))

Сложность: 716 [Версия для печати](#) [Отправить на проверку](#) [Обсуждение на форуме \(37\)](#)

[Все попытки \(15167\)](#) [Все успешные попытки \(3871\)](#) [Рейтинг решений \(2130\)](#)

Рисунок 26. Дано задачи 1057

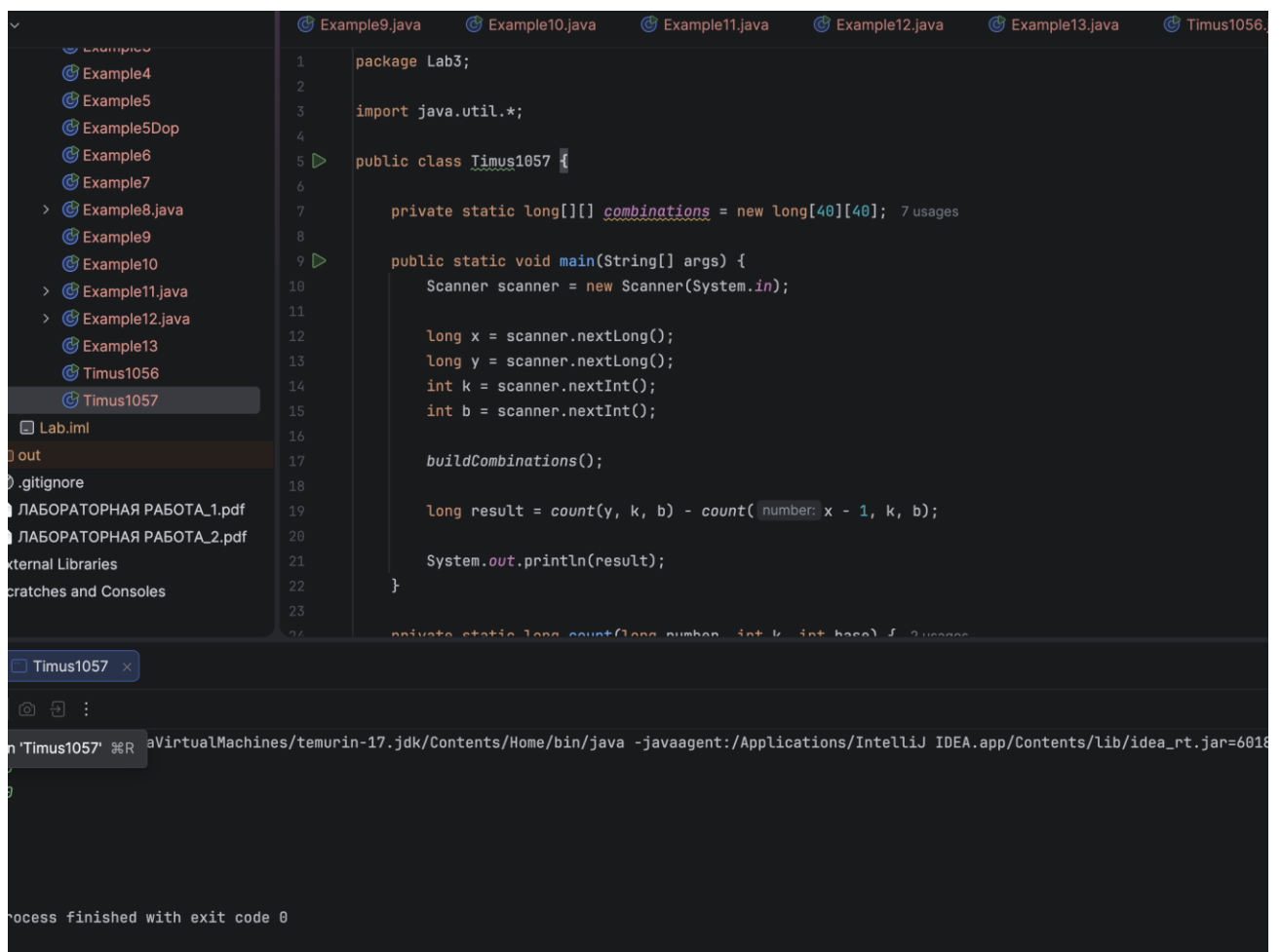


Рисунок 27. Решение задачи 1057

Вывод

В ходе выполнения лабораторной работы были изучены рекурсивные алгоритмы и особенности рекурсивных вызовов в языке Java. Были реализованы программы для вывода рекурсивных последовательностей, вычисления факториала и числа Фибоначчи, перевода числа в двоичную систему счисления, а также рекурсивного ввода и вывода массива.

Также были рассмотрены динамические структуры данных на примере односвязного списка. В работе реализованы способы создания списка с головы и с хвоста, методы добавления, удаления, вставки и рекурсивного вывода элементов. Дополнительно были рассмотрены коллекции Java, выполнено описание ArrayList, LinkedHashSet и LinkedHashMap, а также проведено сравнение времени выполнения операций добавления, удаления и получения элементов. В завершение были решены две задачи с сайта Timus.